
Artificial Neural Networks

- Multi-Layer Perceptron Networks

Unit IV - Topics

- **Multi-Layer Perceptron Networks:** Introduction, MLP with 2 hidden layers, Simple layer of a MLP, Delta learning rule of the output layer, Multilayer feed forward neural network with continuous perceptions, Generalized delta learning rule, Back propagation algorithm.

Multi-Layer Perceptron Networks:

- A Perceptron is the simplest type of neural network consisting of a single layer.
- However, a single-layer perceptron can only solve linearly separable problems (e.g., AND, OR) but fails for non-linearly separable problems (e.g., XOR).
- To overcome this, we use Multi-Layer Perceptrons (MLPs).
- A multilayer perceptron is a type of feedforward neural network consisting of fully connected neurons with a nonlinear kind of activation function.
- It is widely used to distinguish data that is not linearly separable.

Multi-Layer Perceptron Networks:

- An MLP is a type of neural network consisting of at least three layers: an input layer, one or more hidden layers, and an output layer.
- Each neuron in one layer is connected to every neuron in the next layer.
- The MLP learns by adjusting the weights of these connections to minimize the error of its predictions, using a process called backpropagation.

Key features of MLP:

- At least one hidden layer is required.
- Uses non-linear activation functions in hidden layers.
- Can approximate any continuous function (Universal Approximation Theorem).

Multi-Layer Perceptron Networks:

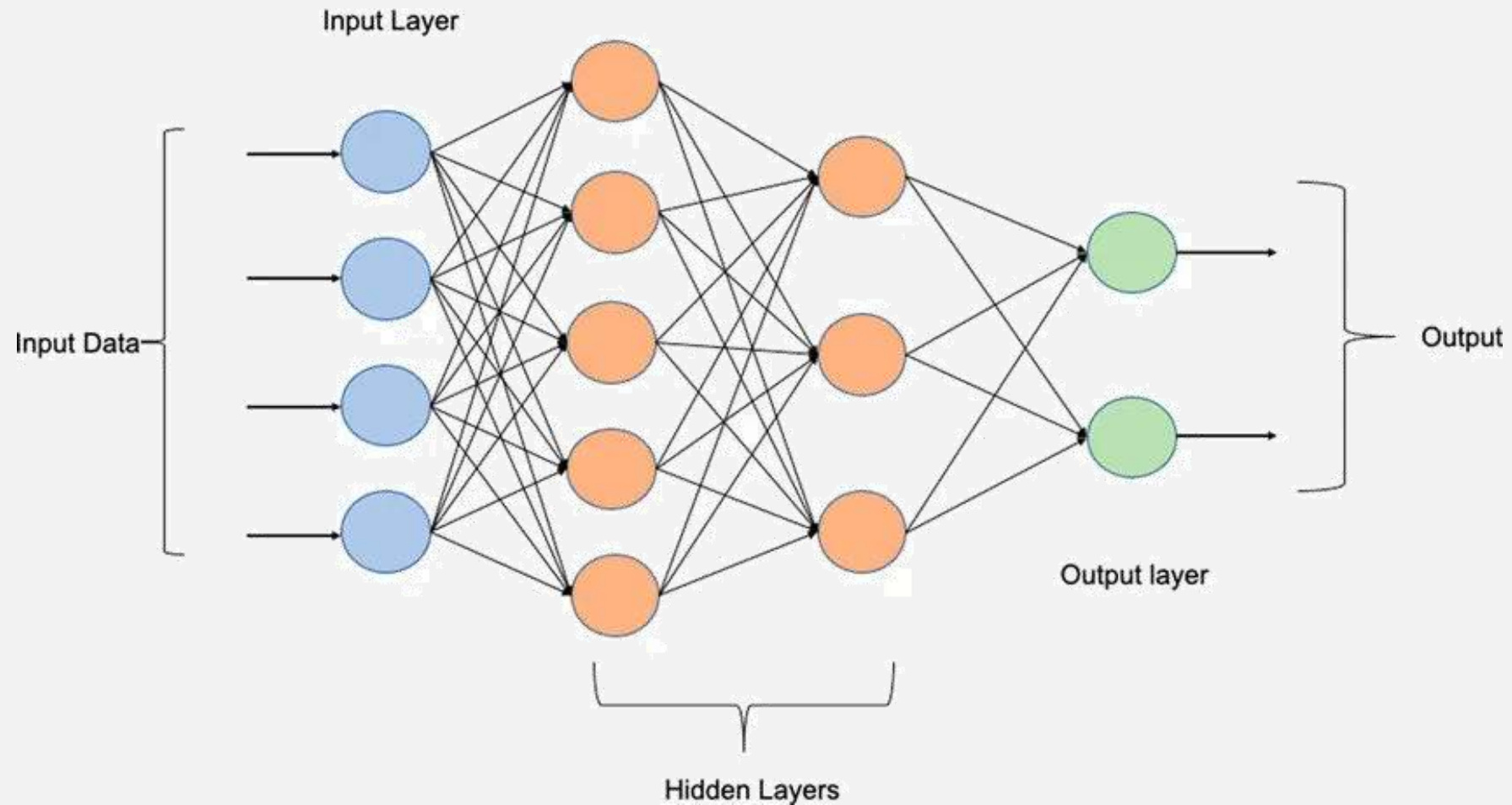


Fig: Diagram of a Multi-Layer Perceptron (MLP) architecture.

Simple layer of a MLP:

A Multi-Layer Perceptron (MLP) is a feed-forward neural network consisting of:

- **Input layer :**

- The input layer consists of nodes or neurons that receive the initial input data.
- This input data could be features extracted from the input samples in a dataset.
- Each neuron in the input layer represents a feature or dimension of the input data.
- For instance, if you have 3 input features the input layer will have three neurons.
- Neurons in the input layer do not perform any computations; they simply pass the input values to the neurons in the first hidden layer.

Simple layer of a MLP:

- **Hidden layer(s):**

- The hidden layers of an MLP consist of interconnected neurons that perform computations on the input data.
- Between the input and output layers, there can be one or more layers of neurons. These layers process the information received from the input layer.
- Each neuron in a hidden layer receives inputs from all neurons in the previous layer (either the input layer or another hidden layer) and produces an output that is passed to the next layer.
- The number of hidden layers and the number of neurons in each hidden layer are hyper parameters depend on the type of problems.

Simple layer of a MLP:

- **Output layer**

- The output layer of an MLP produces the final predictions or outputs of the network.
- The number of neurons in the output layer depends on the task being performed (e.g., binary classification, multi-class classification, regression).
- Each neuron in the output layer receives input from the neurons in the last hidden layer and applies an activation function.
- This activation function is usually different from those used in the hidden layers and produces the final output value or prediction.

MLP with Two Hidden Layers:

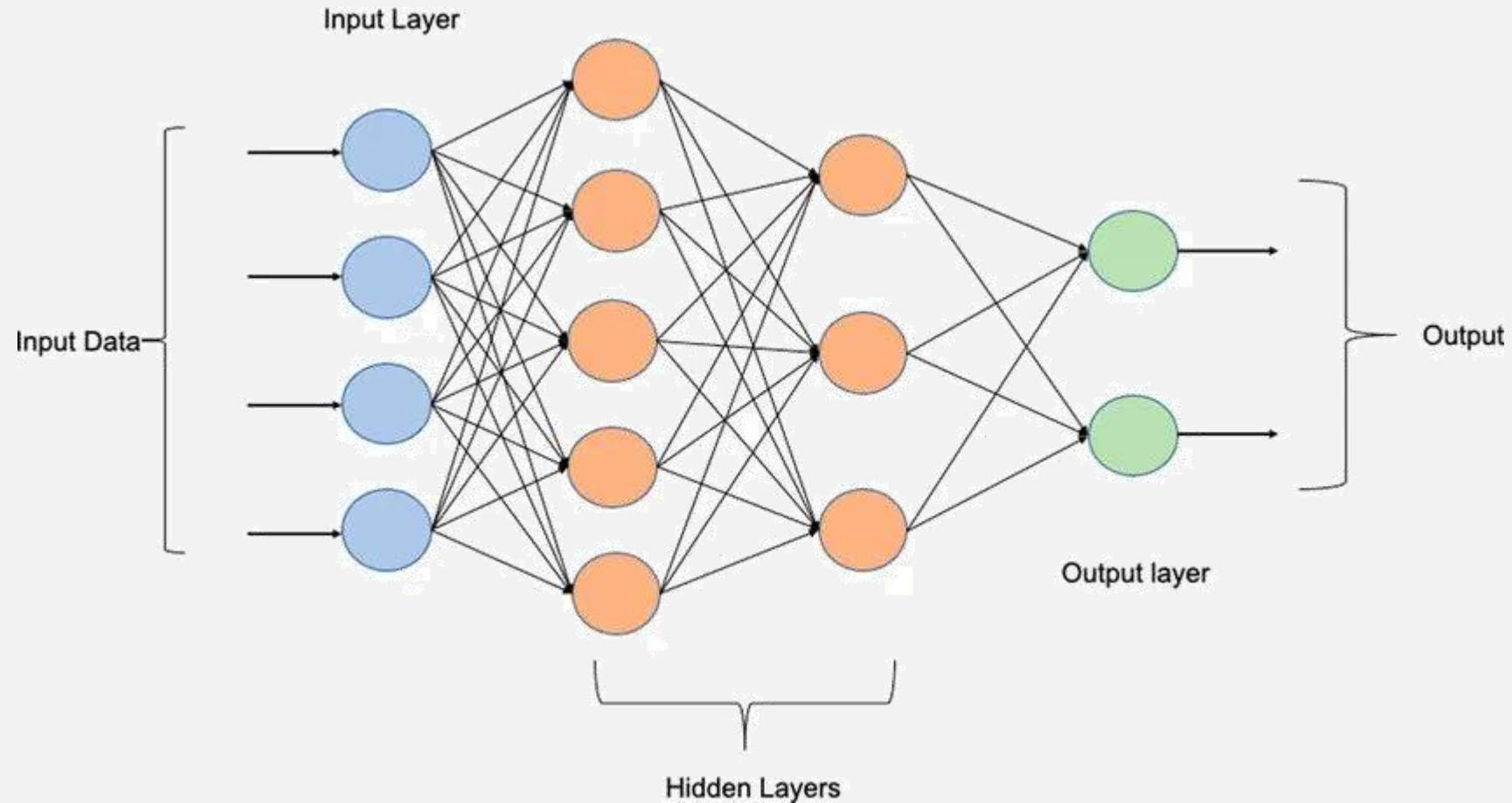


Fig: Diagram of a MLP architecture two hidden layers

MLP with Two Hidden Layers:

- A typical MLP can have one or more hidden layers.
- For example, in a 2-hidden-layer MLP:

Input $\rightarrow$ Hidden Layer 1 $\rightarrow$ Hidden Layer 2 $\rightarrow$ Output $x \rightarrow h_1 \rightarrow h_2 \rightarrow y$

- Each layer performs a transformation:
 - $$h_1 = f(\sum wx + b)$$
 - $$h_2 = f(\sum wx + b)$$
 - $$y = f(\sum wx + b)$$
- More hidden layers increase representational power but also increase computational cost and risk of overfitting.

$f(\cdot)$ = activation function

Multi-Layer Perceptron Networks:

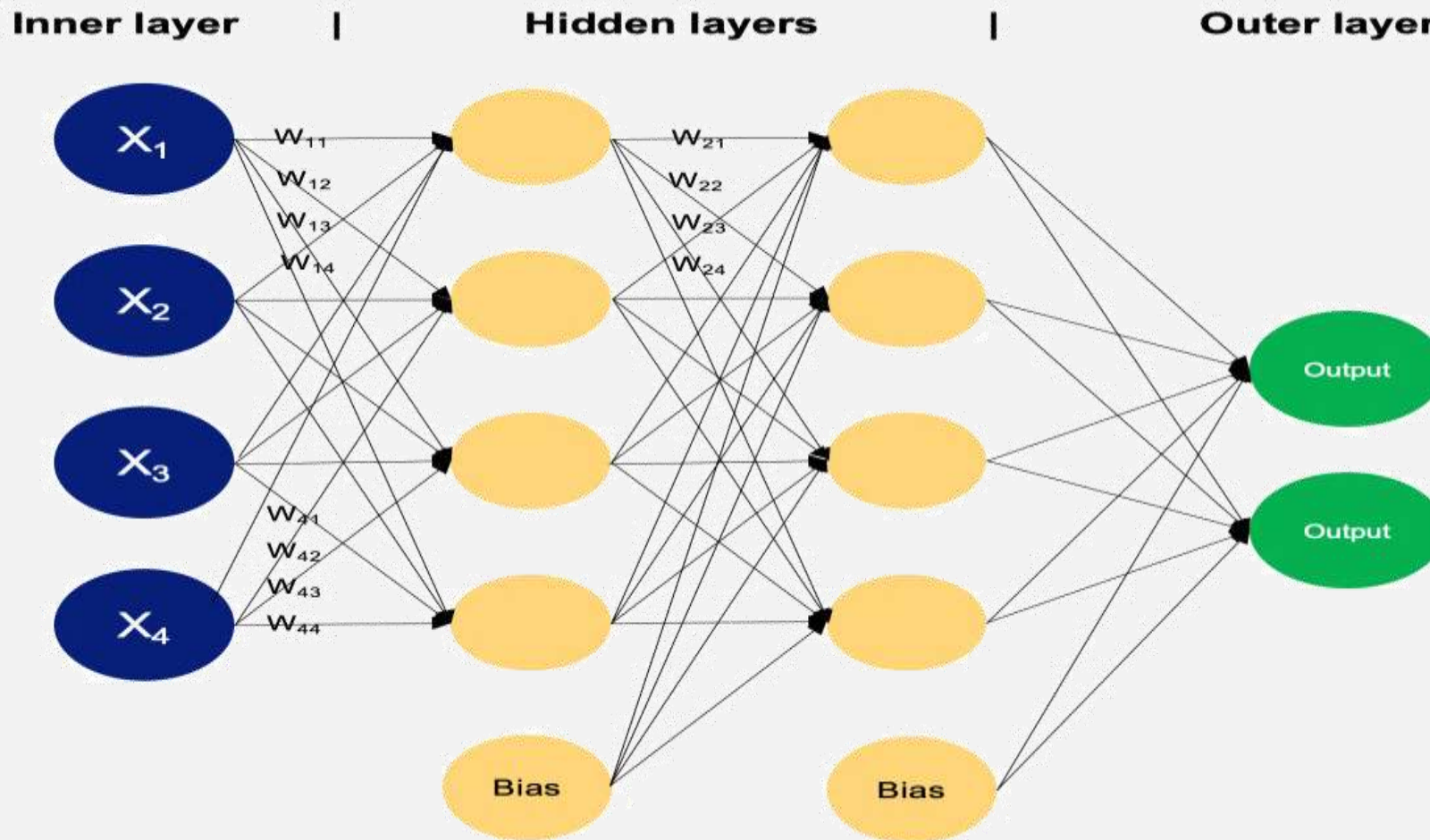


Fig: Multi-Level Perceptron with 2 hidden layers

Multi-Layer Perceptron Networks :

Each layer has the following components:

- **Inputs:** values from previous layer (or raw data in input layer).
- **Weights:** connection strengths between neurons.
- **Bias:** helps shift activation thresholds.
- **Activation function:** introduces non-linearity.
- Each neuron in a hidden layer receives input from all neurons in the previous layer.
- The inputs are multiplied by corresponding weights, denoted as w . The weights determine how much influence the input from one neuron has on the output of another.

Multi-Layer Perceptron Networks :

- In addition to weights, each neuron in the hidden layer has an associated bias, denoted as b .
- The bias provides an additional input to the neuron, allowing it to adjust its output threshold. Like weights, biases are learned during training.

The diagram shows the equation for the output of a neuron m in a hidden layer. The equation is $z_m = f(x_n, w_{mn}) = b + \sum_m x_n w_{mn}$. Red arrows point from text labels to parts of the equation: 'function output' points to z_m ; 'function inputs' points to the inputs of the function f ; 'function name' points to f ; 'bias term' points to b ; 'index for each neuron and matrix row' points to the summation index m ; 'element n of feature vector $\mathbf{x}$ ' points to x_n ; and 'element mn of feature matrix $\mathbf{W}$ ' points to w_{mn} .

$$z_m = f(x_n, w_{mn}) = b + \sum_m x_n w_{mn}$$

function output

function inputs

function name

bias term

index for each neuron and matrix row

element n of feature vector $\mathbf{x}$

element mn of feature matrix $\mathbf{W}$

Multi-Layer Perceptron Networks :

- The weighted sum is then passed through an activation function, denoted as a .
- The activation function introduces nonlinearity into the network, allowing it to learn and represent complex relationships in the data.
- The choice of activation function depends on the nature of the task and the desired properties of the network.

The diagram shows the equation $a_m = \sigma(z_m) = \frac{1}{1 + e^{-z_m}}$ with several annotations in red text and arrows:

- An arrow points from the text "function output" to a_m .
- An arrow points from the text "sigmoid function symbol (sigma)" to σ .
- An arrow points from the text "linear function output becomes sigmoid input" to z_m .
- An arrow points from the text "Euler's number symbol (-2.71828)" to e .
- A curved arrow points from the text "linear function output becomes sigmoid input" to the entire fraction $\frac{1}{1 + e^{-z_m}}$.

Multi-Layer Perceptron Networks :

Output function

- For binary classification problems each output unit implements a threshold function as:
$$\hat{y} = f(a_m) = \begin{cases} +1 & \text{if } a > 0.5 \\ -1 & \text{otherwise} \end{cases}$$

- For regression problems (problems that require a real-valued output value like predicting income or test-scores) each output unit implements an identity function as::
$$\hat{y} = f(a_m) = a_m$$

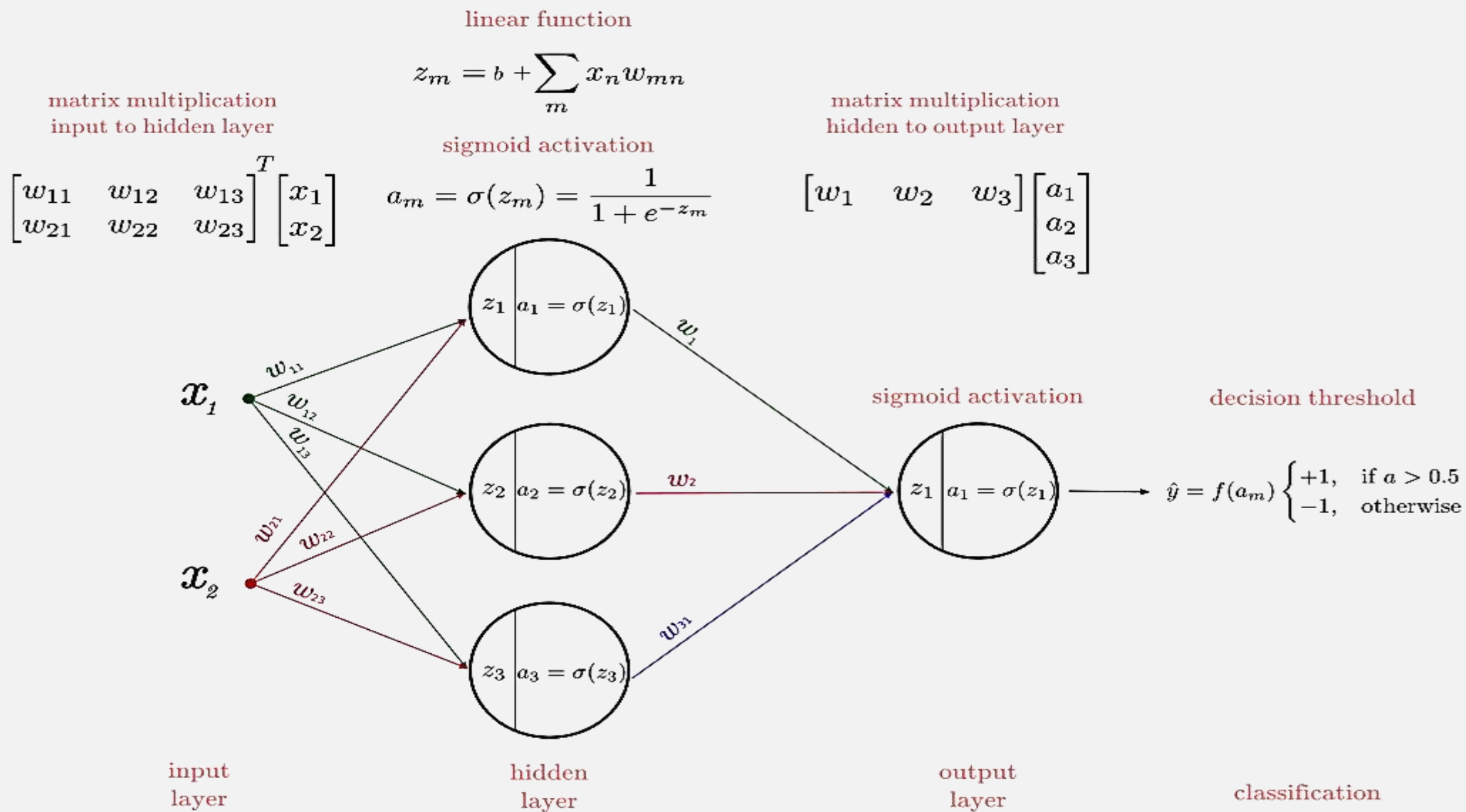
- For multiclass classification problems, we can use a softmax function as:

$$\text{softmax}(\mathbf{z}_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Multi-Layer Perceptron Networks :

All neural networks can be divided into two parts:

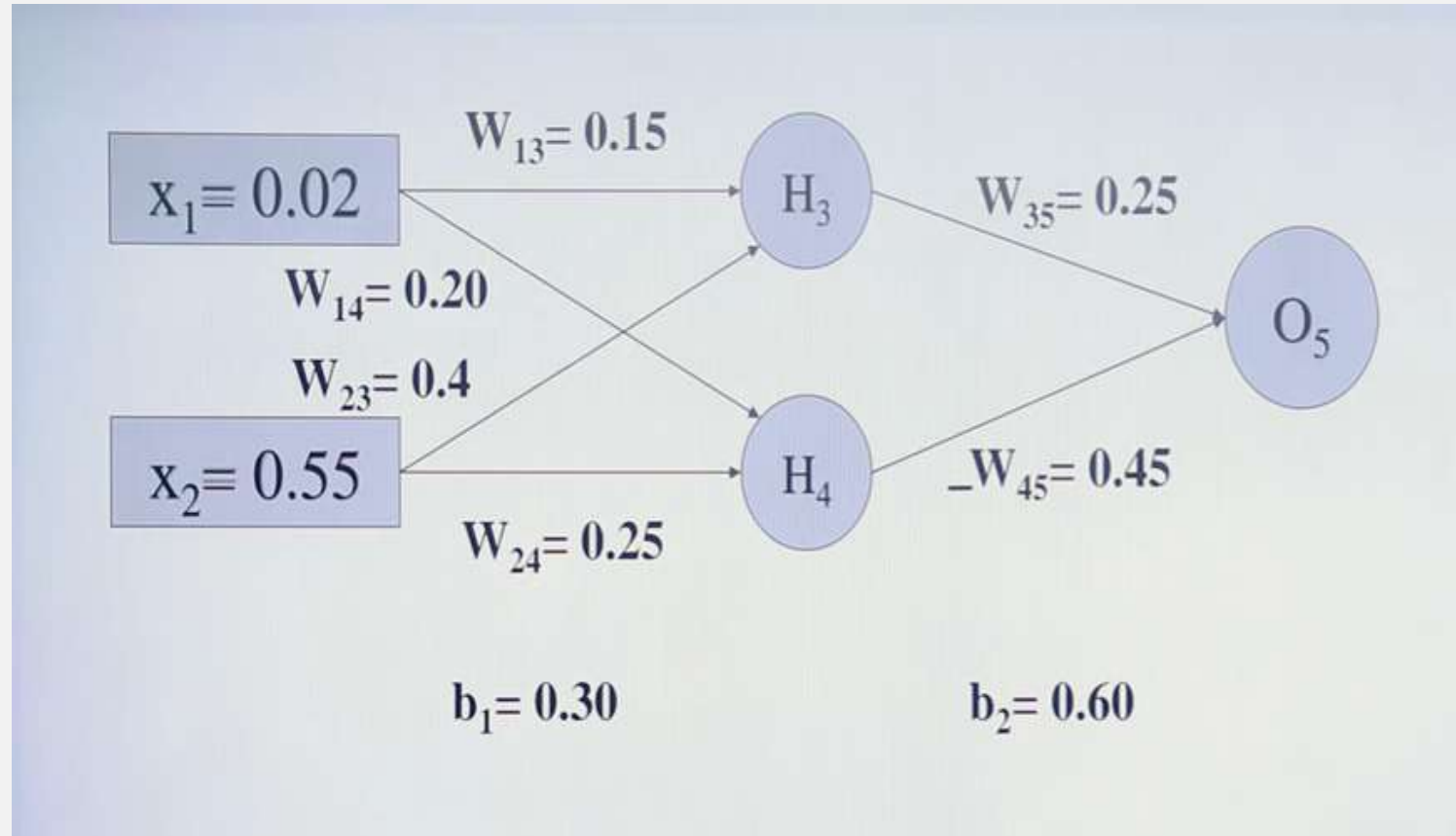
- A forward propagation phase, where the information “flows” forward to compute predictions and the error.
- The backward propagation phase, where the backpropagation algorithm computes the error derivatives and update the network weights.



Information flow forward pass neural network with one hidden layer

Multi-Layer Perceptron Networks :

Q1: For the given Multi-Layer Perceptron shown in the figure, perform the forward propagation to calculate the output for tanh activation function on each layer.



Multi-Layer Perceptron Networks :

Cost function:

- People sometimes call it loss function, or error function. Conventionally, loss function usually refers to the measure of error for a single training case, cost function to the aggregate error for the entire dataset.
- Nowadays, you would probably want to use different cost functions for different types of problems.
- In their original work, Rumelhart, Hinton, and Williams used the sum of squared errors defined as:

Multi-Layer Perceptron Networks :

The diagram shows the Mean Squared Error (MSE) formula with several annotations in red text and arrows:

- sum of squared errors for the entire dataset**: Points to the variable E .
- added to simplify gradient computation**: Points to the coefficient $\frac{1}{2}$.
- index for input-output pair**: Points to the summation index k .
- predicted value for training example k** : Points to the term $\hat{y}$.
- expected value for training example k** : Points to the term y .

$$E = \frac{1}{2} \sum_k (y - \hat{y})^2$$

$y = \text{actual output}$

$\hat{y} = \text{Predicted Output}$

partial derivative of the error w.r.t. the weight

partial derivative symbol (stylized d)

partial derivative of the error w.r.t. sigmoid activation

partial derivative of linear function w.r.t. the weight

index for the current layer

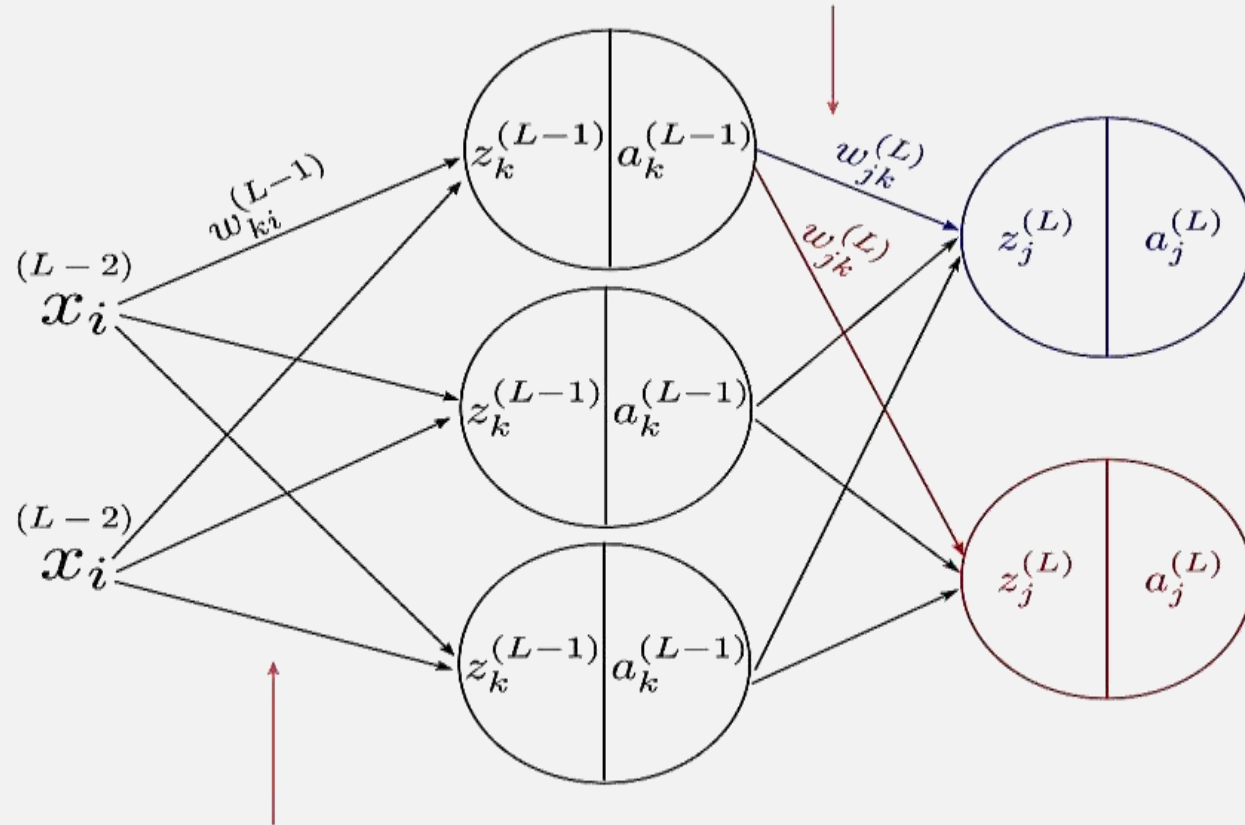
partial derivative of sigmoid activation w.r.t. linear function

$$\frac{\partial E}{\partial w^{(L)}} = \frac{\partial E}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z^{(L)}} \times \frac{\partial z^{(L)}}{\partial w^{(L)}}$$

$$z^{(L)} \text{ neuron output at output layer} = \sum wx + b$$

derivative of the error w.r.t. weights in (L)

$$\frac{\partial E_i}{\partial w_{jk}^{(L)}} = \frac{\partial E_i}{\partial a_j^{(L)}} \times \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \times \frac{\partial z_j^{(L)}}{\partial w_{jk}^{(L)}}$$



$$E = \frac{1}{2} \sum_j (y - \hat{y})^2$$

$$\frac{\partial E}{\partial w_{ki}^{(L-1)}} = \left(\sum_j \frac{\partial E}{\partial a_j^{(L)}} \times \frac{\partial a_j^{(L)}}{\partial z_j^{(L)}} \times \frac{\partial z_j^{(L)}}{\partial a_k^{(L-1)}} \right) \times \frac{\partial a_k^{(L-1)}}{\partial z_k^{(L-1)}} \times \frac{\partial z_k^{(L-1)}}{\partial w_{ki}^{(L-1)}}$$

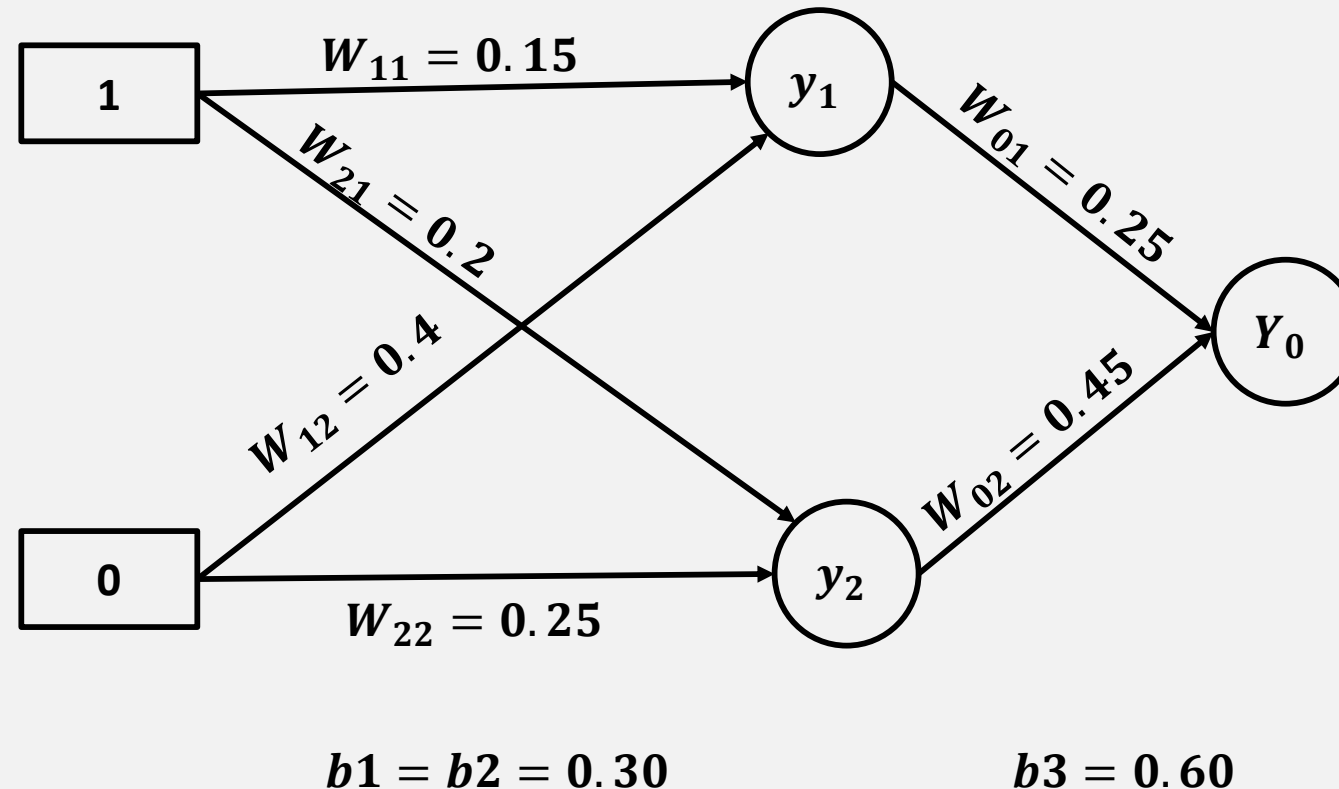
derivative of the error w.r.t. weights in (L-1)

Delta Learning Rule of the Output Layer:

- The Delta Rule (Widrow-Hoff Rule) updates weights to minimize error between desired and actual output.
- Used to update weights of output neurons
- Based on gradient descent minimization of error.
- Training MLP requires error measurement. Most common: Mean Squared Error (MSE)

Delta Learning Rule of the Output Layer:

Question: Consider an MLP with 2 inputs, 1 hidden layer of 2 neurons, and 1 output neuron (sigmoid activation, $t=1$). Using given inputs, initial weights, and bias, perform one iteration of the delta learning rule to update only the output weights and bias.



Multilayer Feed-Forward Neural Network with Continuous Perceptrons:

- In practice, neurons in MLP use continuous activation functions (sigmoid, tanh, ReLU) instead of step functions.
- Step functions are not differentiable, preventing gradient descent from working.
- Continuous functions allow smooth weight updates
- Feed-forward computation:
 - Inputs are multiplied with weights, biases added, and passed through activation functions layer by layer until the output is produced.

Multilayer Feed-Forward Neural Network with Continuous Perceptrons:

- Continuous functions allow differentiability, which is essential for training using gradient descent.
- modern MLPs are multilayer feed-forward networks with continuous activation functions.

Generalized Delta Learning Rule:

- Extension of the Delta Rule to multilayer networks.
- Instead of only updating the output layer, errors are propagated backward into hidden layers.
- The error for hidden layer neurons is computed using:

$$\delta_j = f'(net) \sum_k \delta_k w_{jk}$$

Where: δ_j : *error hidden neuron j*

$f'(net)$: derivative of activation function

$\sum_k$: sum over errors of neurons in the next layer

- This is the basis of backpropagation.

Backpropagation Algorithm:

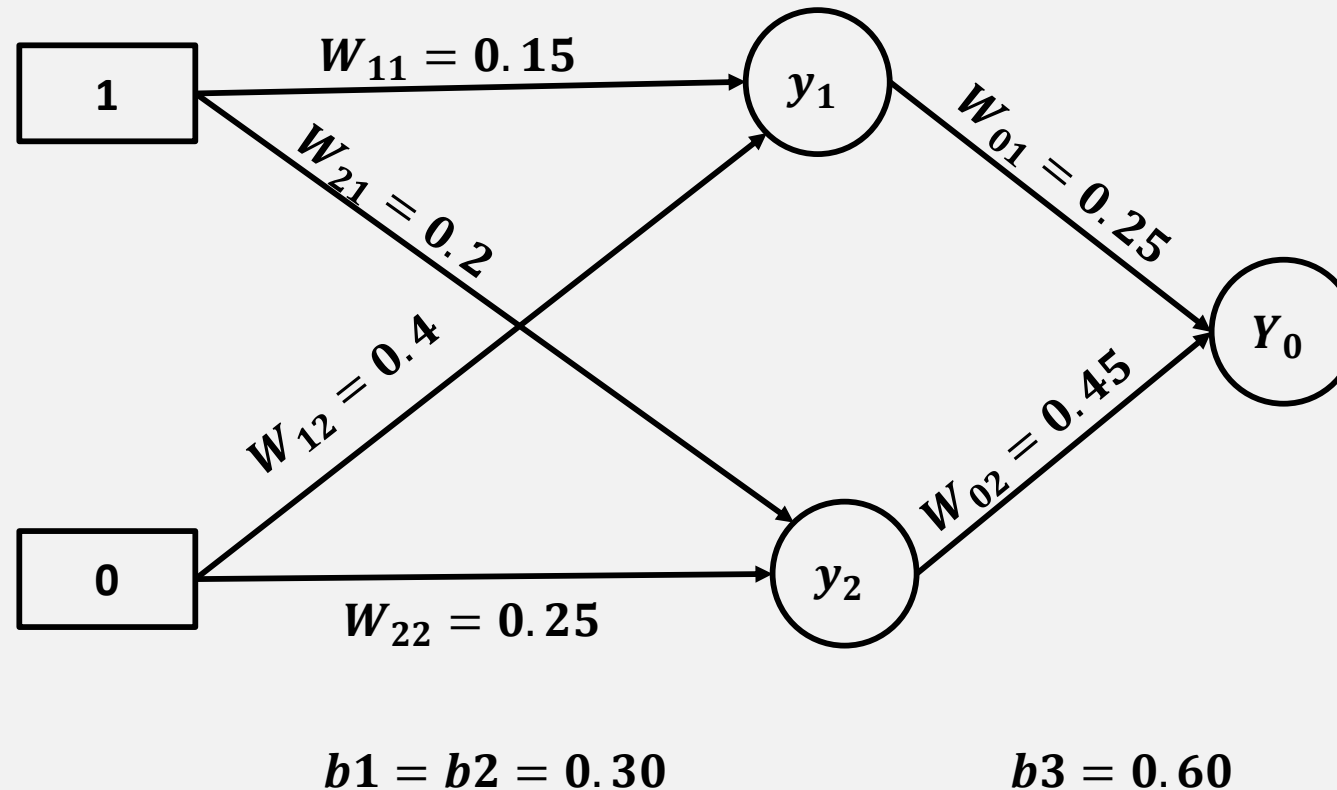
- Most widely used algorithm for training MLPs.
- Steps: Forward pass – compute output using current weights.
Error calculation – difference between target and output.
Backward pass – propagate errors back through the network.
Weight update – using gradient descent and generalized delta rule.
- Backpropagation uses chain rule of calculus to compute derivatives.
- Weight update formula:

$$\Delta w_{ij} = -\eta \frac{\partial E}{\partial w_{ij}}$$
$$w_{ij} (new) = w_{ij} (old) + \Delta w_{ij}$$

Where: E is the error function (often Mean Squared Error)

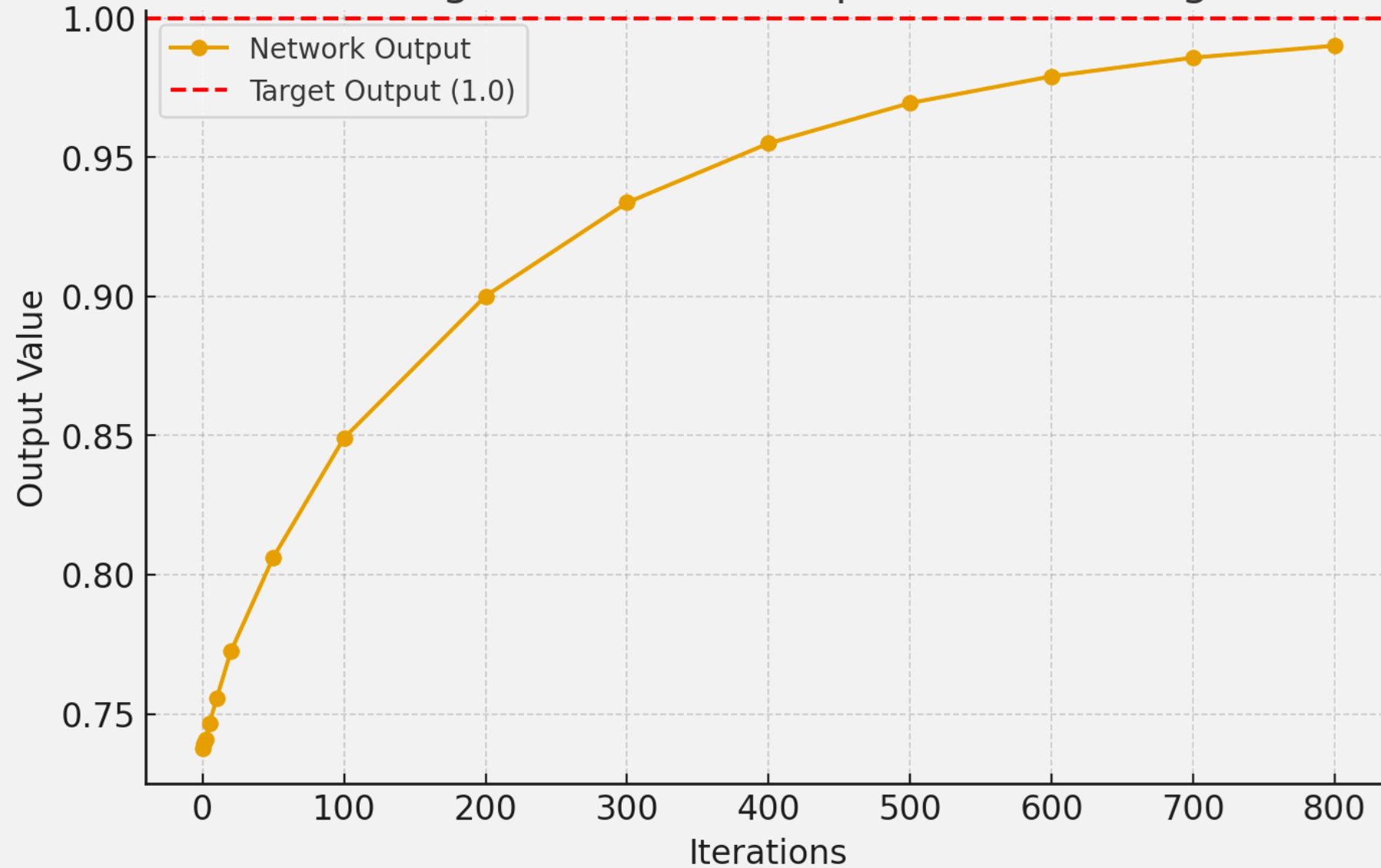
Backpropagation Algorithm :

Question: Consider an MLP with 2 inputs, 1 hidden layer of 2 neurons, and 1 output neuron (sigmoid activation, $t=1$, $\eta=0.1$). Using given inputs, initial weights, and bias, perform one iteration of the delta learning rule to update the weights and bias of networks.



Iteration	Network Output ($\hat{y}$)	Error $\left(E = \frac{1}{2}(y - \hat{y})^2\right)$
0 (initial)	0.7375	0.0345
1	0.7392	0.0340
2	0.7408	0.0335
5	0.7466	0.0320
10	0.7555	0.0297
20	0.7724	0.0260
50	0.8062	0.0188
100	0.8491	0.0115
200	0.9001	0.0050
300	0.9337	0.0022
400	0.9550	0.0010
500	0.9695	0.0005
600	0.9791	0.0002
700	0.9858	0.0001
800	0.9901	<0.00005

Convergence of MLP Output towards Target



Backpropagation: It's an algorithm to compute gradients efficiently.

- Given a neural network and a loss function, backpropagation applies the chain rule to compute $\partial L / \partial w$ for each weight w .
- Output: the gradients (but does not update weights itself).
- Gradient Descent It's: an optimization algorithm that uses the gradients (from backprop) to update the weights: $w \leftarrow w - \eta \cdot \partial L / \partial w$ where η = learning rate.
- Output: new weight values after each step.
- Backpropagation calculates the gradients.

Lab exp:

Implement a multilayer perceptron (MLP) trained using backpropagation to solve the XOR problem. Compare the performance when using different activation functions (Sigmoid, Tanh, ReLU).

Multi-Layer Perceptron Networks :